COMPILER DESIGN LABORATORY (BCSE307P)

Fall Semester 2026–27

SQL2Mongo

A SQL-to-MongoDB Query Compiler and Execution Engine

Project Report

Submitted by

Name: Alan James

Register Number: 24BCT0181

October 2026

Abstract

Most developers learn SQL before they ever work with MongoDB, and moving to MongoDB means relearning how to write the same queries in a very different language. SQL2Mongo addresses this. It is a source-to-source compiler, written in C99, that translates SQL `SELECT` queries into MongoDB aggregation pipelines and runs them on an in-memory document store.

The compiler implements seven phases: lexical analysis, recursive-descent parsing with parse-tree and AST output, symbol-table-driven semantic analysis, intermediate code in three-address code, quadruple and triple form, an optimization pass, MongoDB Query Language (MQL) code generation, and execution.

The supported language covers projections, `WHERE` filters with `AND`, `OR` and parentheses, six comparison operators, `ORDER BY` and `LIMIT`. Each phase can be shown on its own through a command-line flag, so every intermediate form is visible during a demonstration.

The compiler is 2,370 lines of C in ten modules and uses only the C standard library. Of 19 test queries covering valid, boundary and erroneous input, 17 behave exactly as expected; the remaining 2 expose limitations in the generated target code, which are documented along with their fixes.

Contents

Abstract **i**

1 Introduction **1**

1.1 Background 1

1.2 Motivation 1

1.3 Problem Area 1

2 Problem Statement **2**

2.1 Requirements 2

3 Objectives **3**

3.1 Scope 3

4 Literature Survey **4**

4.1 Existing Approaches 4

4.2 SQL-to-MQL Mapping 4

4.3 Compiler Concepts Studied 5

4.4 Technology Selection 5

5 System Design **6**

5.1 Architecture 6

5.2 Module Description 7

5.3 Data Flow 7

5.4 Workflow 8

6 Methodology **9**

6.1 Approach 9

6.2 Development Phases 10

7 Compiler Design Concepts Used **11**

7.1 Lexical Analysis 11

7.2 Syntax Analysis 11

7.3 Abstract Syntax Tree 13

7.4 Symbol Table and Semantic Analysis 13

7.5 Intermediate Code Generation 14

7.6 Code Optimization . . . 15
7.7 Target Code Generation . . . 15
7.8 Interpretation and Execution . . . 16
7.9 Error Detection and Recovery . . . 16

8 Implementation **17**
8.1 Tools Used . . . 17
8.2 Modules . . . 18
8.3 Key Algorithms . . . 18
8.4 Project Structure . . . 19
8.5 User Instructions . . . 19

9 Testing and Results **21**
9.1 Test Cases . . . 21
9.2 Output Evidence . . . 22
9.3 Performance . . . 23
9.4 Findings . . . 24

10 Conclusion **25**

11 Future Enhancements **26**

References **27**

A Code Excerpts and Sample Data **28**
A.1 Operator Precedence in the Parser . . . 28
A.2 Post-order Three-Address Code for Filters . . . 28
A.3 Sample Data . . . 29

Chapter 1: Introduction

SQL2Mongo lets someone who already knows SQL write a query in SQL and get the equivalent MongoDB query. It does this by applying the full compiler pipeline to the two languages, with SQL as the source and the MongoDB Query Language (MQL) as the target.

1.1 Background

Relational databases are queried with SQL, a declarative language with a fixed keyword grammar. Document databases such as MongoDB store JSON-like documents and are queried with MQL, where a query is itself a JSON structure. MongoDB's aggregation framework expresses a query as a pipeline of stages such as `$match`, `$sort`, `$limit` and `$project` [6].

Both languages describe the same operations, but with different syntax and structure, so moving a query from one to the other is a translation problem.

1.2 Motivation

Most students and developers learn SQL first. It is what database courses teach, and it is the language people reach for when they think about querying data. Many of them later work on projects that use MongoDB, and they then have to learn a second, very different way of writing the same queries.

I faced this myself during my internship, where the project used MongoDB. I already knew how to write the queries I needed in SQL, but I had to work out how to express each one in MongoDB: which operator replaces `>=`, how `AND` and `OR` turn into nested `$and` and `$or` arrays, and in what order the pipeline stages must go. A small mistake in this translation does not raise an error. The query simply returns the wrong documents, or none at all.

This project grew out of that experience. Translating one query language into another is exactly what a compiler does, so I built one that takes the SQL a developer already knows and produces the equivalent MongoDB query. Because every phase prints its output, the tool also shows how each part of the SQL query maps to MongoDB, which helps someone who is learning MongoDB as well as someone who only needs the translated query.

1.3 Problem Area

The project sits in source-to-source compilation (transpilation) and database query processing. It combines classic front-end phases (scanning, parsing, semantic checks) with query-processing ideas (relational operators, query plans, predicate pushdown).

Chapter 2: Problem Statement

A developer who knows SQL but has to work with MongoDB must translate every query by hand. The problem this project solves is: given a SQL `SELECT` query over a known schema, produce an equivalent and valid MongoDB aggregation pipeline, or report exactly where and why the query is invalid.

Hand translation from SQL to MQL has three recurring failure points:

- **Operator mapping.** Each SQL comparison has a different MQL spelling (`>=` to `$gte`, `!=` and `<>` to `$ne`), and equality is written without an operator.
- **Logical structure.** Operator precedence must be preserved: `a AND b OR c` means `(a AND b) OR c`, which becomes an `$or` array containing an `$and` array.
- **Schema errors found late.** A misspelled field or a string compared with a number is not rejected by MongoDB. The query simply returns no documents, so the mistake goes unnoticed.

The translator must therefore check queries before generating code, and its diagnostics must point to the line and column of the fault.

2.1 Requirements

Type	Requirement
Input	A SQL query, either as a `.sql` file path or as an inline string on the command line.
Output	Token table, parse tree, AST, symbol table, TAC, quadruples, triples, optimization report, MQL pipeline and query results, each selectable by a flag.
Functional	Reject lexical, syntax and semantic errors with a message giving the line, column and expected construct.
Functional	Return exit code 0 on success, 1 on a lexical or syntax error and 2 on a semantic error, so tests can be automated.
System	Any machine with a C99 compiler; no database server or third-party library is needed.

Table 2.1: System requirements

Chapter 3: Objectives

1. Build a lexical analyser that recognises 11 SQL keywords, identifiers, integer, float and string literals, 7 comparison lexemes and 5 delimiters, skips comments, and tracks line and column.
2. Build a recursive-descent parser for the query grammar that prints a concrete parse tree and builds an abstract syntax tree (AST).
3. Build a symbol table of collections and typed fields, and use it to check collection existence, field existence and type compatibility.
4. Generate intermediate code as three-address code, quadruples and triples.
5. Apply query optimizations: predicate pushdown and projection pruning.
6. Generate target MQL as a `db.<collection>.aggregate([...])` pipeline.
7. Execute the compiled query on an in-memory document store to show the translation gives the right results.
8. Validate the system with valid, invalid, boundary and complex test queries.

3.1 Scope

In scope: single-collection `SELECT` queries with a column list or `*`; `WHERE` with `=`, `!=`, `<>`, `>`, `>=`, `<`, `<=`, `AND`, `OR` and parentheses; `ORDER BY` on one field, `ASC` or `DESC`; `LIMIT`; case-insensitive keywords.

Out of scope: `JOIN`, `GROUP BY` and aggregate functions, subqueries, `IN`, `LIKE` and `BETWEEN`, and data-changing statements (`INSERT`, `UPDATE`, `DELETE`). `NOT` is recognised by the lexer as a reserved word but is not yet part of the grammar.

Chapter 4: Literature Survey

SQL-to-MQL translators already exist, but they hide their internals behind a product or a third-party parser. SQL2Mongo builds every phase by hand so each one can be shown and explained.

4.1 Existing Approaches

Approach	How it works	Difference from SQL2Mongo
MongoDB SQL to Aggregation Mapping Chart [5]	Official reference table that maps SQL clauses to pipeline stages.	A manual guide; it performs no translation or checking.
mongotranslate, MongoDB BI Connector [9]	Command-line tool that shows how a SQL query is expressed as an aggregation pipeline, using a schema (DRDL) file.	Part of a large commercial product; its phases are not visible.
sql-to-mongo-db-query-converter [10]	Java library that converts SQL into MongoDB find, update, delete and aggregation queries.	Parsing is delegated to the JSqlParser library, so there is no hand-built lexer or parser.
SQL2Mongo (this project)	Hand-written lexer, parser, semantic analyser, IR generator, optimizer, code generator and executor in C.	Every phase prints its own output; schema errors are caught before code generation.

Table 4.1: Comparison with existing SQL-to-MongoDB translators

4.2 SQL-to-MQL Mapping

The target forms follow MongoDB's own mapping chart [5].

SQL construct	MQL stage or operator
`FROM t`	`db.t.aggregate([...])`
`WHERE`	`$match`
`ORDER BY f ASC / DESC`	`$sort: { f: 1 } / { f: -1 }`
`LIMIT n`	`$limit: n`
`SELECT a, b`	`$project: { _id: 0, a: 1, b: 1 }`
`=`	implicit equality `{ f: v }`
`!=, <>, >, >=, <, <=`	`$ne`, `$ne`, `$gt`, `$gte`, `$lt`, `$lte`
`AND, OR`	`$and: [...], $or: [...]`

Table 4.2: SQL constructs and their MQL equivalents

4.3 Compiler Concepts Studied

- **Phases of a compiler.** The project follows the analysis–synthesis model of Aho, Lam, Sethi and Ullman [1]: scanning, parsing, semantic analysis, intermediate code, optimization and code generation.
- **Recursive-descent parsing.** One function per non-terminal; operator precedence is encoded by grammar layering (`OR` below `AND` below comparisons).
- **Symbol tables and type checking.** The schema catalog acts as a symbol table holding each field's name and type.
- **Intermediate representations.** Three-address code, quadruples and triples, with relational-algebra operators [2] (scan, filter, sort, limit, project) as the instruction set.
- **Query optimization.** Filtering early (predicate pushdown) is a standard relational technique [3]. MongoDB's pipeline optimization guide [7] also moves `$match` ahead of `$sort` and keeps `$project` last.

4.4 Technology Selection

C was chosen from the course's recommended languages because it forces every data structure to be explicit [4]. The lexer and parser are written by hand rather than with Flex and Bison, so the scanning and parsing algorithms are visible in the source. Development used GCC (MinGW 6.3.0), Visual Studio Code, Git for version history and a PowerShell script for automated tests.

Chapter 5: System Design

SQL2Mongo is a single-pass pipeline of seven phases. The parser builds one AST, and every later phase reads it.

5.1 Architecture

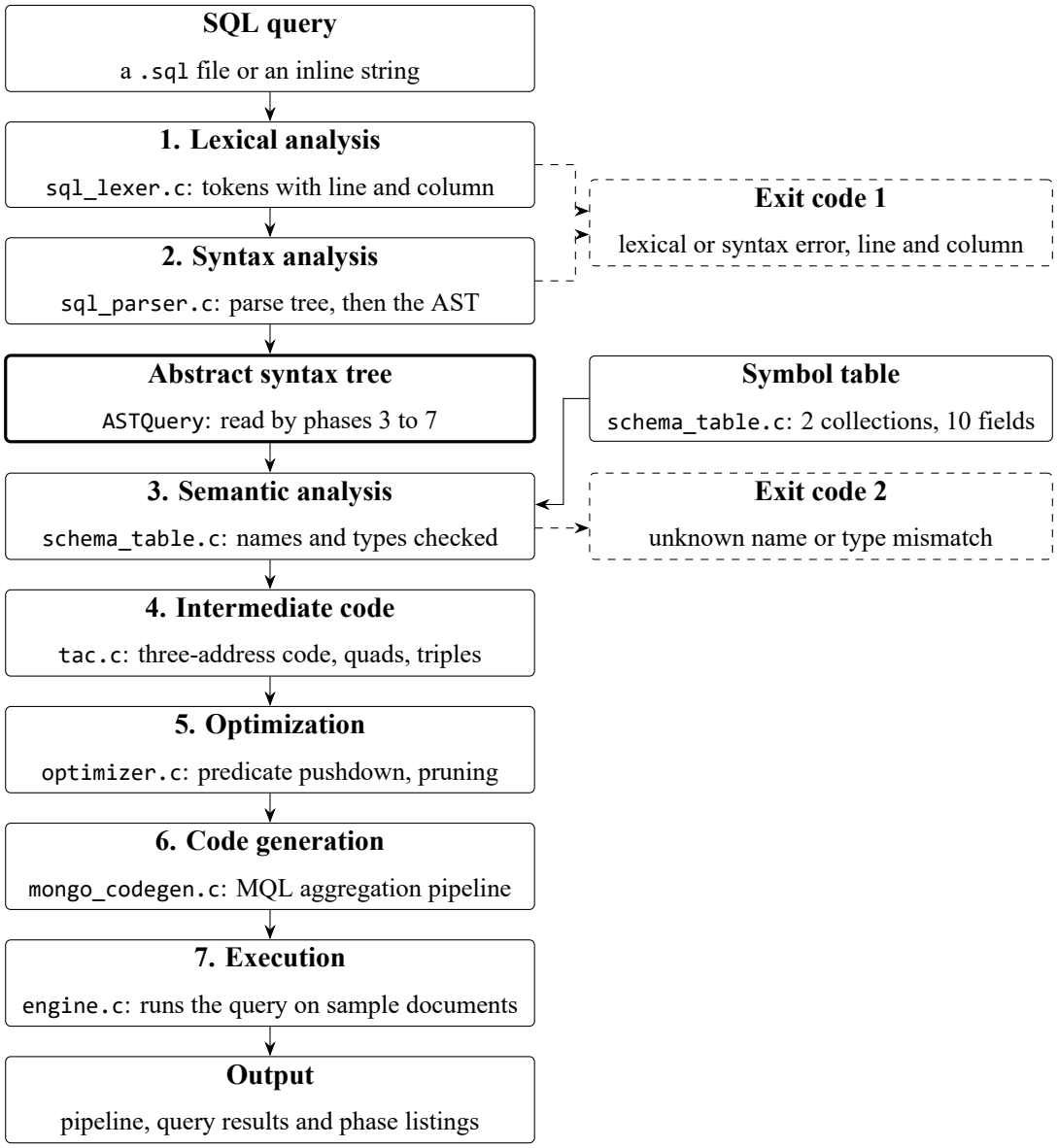


Figure 5.1: Architecture of SQL2Mongo (dashed lines show error exits)

The phases run in the order set by `main.c`. The symbol table feeds only semantic analysis. Lexical and syntax errors exit with code 1 and semantic errors with code 2, so later phases never see an invalid query.

5.2 Module Description

The ten modules form three groups, coordinated by `main.c`:

- **Front end:** `sql_lexer`, `sql_parser` and `sql_ast` turn text into a checked syntax tree.
- **Middle:** `schema_table`, `tac`, `optimizer` and `query_plan` check meaning and build the intermediate forms.
- **Back end:** `mongo_codegen` writes the MQL pipeline and `engine` executes the query.

Section 8.2 lists each module's size and main functions.

5.3 Data Flow

Phase	Reads	Produces
1. Lexical analysis	Query text	Tokens (type, lexeme, value, line, column)
2. Syntax analysis	Tokens, pulled one at a time from the lexer	Parse tree listing and the AST
3. Semantic analysis	AST and symbol table	Pass, or a semantic error
4. Intermediate code	AST	Three-address code, quadruples, triples
5. Optimization	AST and IR	Optimization report
6. Code generation	AST	MQL `aggregate` pipeline
7. Execution	AST and the collection's documents	Result documents as JSON

Table 5.1: Data flow between phases

The token table printed by `--tokens` comes from a separate display pass. During parsing, the parser asks the lexer for each next token on demand.

5.4 Workflow

1. The user runs `sql2mongo.exe` with a query file or an inline query, plus optional phase flags.
2. `main.c` reads the input and loads the symbol table.
3. The phases run in order. Each prints its listing when its flag, or `--all`, is set.
4. On the first error, the compiler prints the diagnostic and exits with code 1 or 2.
5. Otherwise it prints the MQL pipeline and the query results, and exits with code 0.

Chapter 6: Methodology

The compiler was built as a single-pass, phase-by-phase pipeline around one shared data structure, the AST, with every phase observable on its own.

6.1 Approach

1. **Specify the language first.** Tokens, keywords, operators and the grammar (Chapter 7) were fixed before any code was written, so the lexer and parser had an exact target.
2. **One module per phase.** Each phase is a `.c`/`.h` pair with a small public interface, for example `lexer_next_token`, `parser_parse`, `schema_validate_query` and `tac_generate`.
3. **AST as the contract between phases.** The parser produces an `ASTQuery`; every later phase reads it. This keeps phases independent and lets any of them be tested alone.
4. **Fail fast with phase-specific diagnostics.** Compilation stops at the first error. The message names the phase, the line and column, and what was expected. The exit code tells the phase: 1 for lexical or syntax, 2 for semantic.
5. **Make every phase visible.** Each phase has a print function and a command-line flag (`--tokens`, `--tree`, `--ast`, `--schema`, `--tac`, `--quads`, `--triples`, `--opt`, `--mongo`, `--run`), so a reviewer can inspect any intermediate form.
6. **Verify by execution.** The execution engine evaluates the same AST against sample documents. Its results confirm that the generated pipeline means what the SQL query means.
7. **Test with expected exit codes.** Each test query has a known correct exit code, and a PowerShell script runs them all and reports pass or fail.

6.2 Development Phases

Phase	Focus	Outcome
Phase 1: Problem definition and design	Topic, problem statement, language specification, architecture	Lexer and parser prototype with token table and validity check
Phase 2: Core implementation	Symbol table, semantic checks, AST, three-address code	Working front end and intermediate code generator
Phase 3: Final implementation and testing	Quadruples, triples, optimizer, MQL generator, execution engine, test suite	Complete seven-phase compiler with automated tests

Table 6.1: Development phases

Chapter 7: Compiler Design Concepts Used

SQL2Mongo uses eight core compiler concepts, one per phase. The running example in this chapter is the query below (test case 1).

```
SELECT name, age FROM students WHERE age >= 20 ORDER BY age DESC LIMIT 3;
```

7.1 Lexical Analysis

The scanner (`sql_lexer.c`) turns the character stream into tokens using maximal munch with one character of lookahead. The lookahead (`peek_next`) separates `>` from `>=`, `<` from `<=` and `<>`, and `-` from a `--` comment. Each token records its type, lexeme, numeric value, line and column.

Token class	**Pattern or lexemes**	**Token type**
Keywords (11, case-insensitive)	`SELECT FROM WHERE ORDER BY ASC DESC LIMIT AND OR NOT`	`TOK_SELECT` … `TOK_NOT`
Identifier	`[A-Za-z_][A-Za-z0-9_]*`, not a keyword	`TOK_IDENT`
Number literal	`[0-9]+(\.[0-9]*)?`	`TOK_NUMBER`
String literal	`'…'` or `"…"`, with `\` escapes	`TOK_STRING`
Comparison operator	`=  !=  <>  >  >=  <  <=`	`TOK_EQ`, `TOK_NEQ`, `TOK_GT`, `TOK_GTE`, `TOK_LT`, `TOK_LTE`
Delimiter	`,  ;  *  (  )`	`TOK_COMMA` … `TOK_RPAREN`
Comment (discarded)	`--` … to end of line, `/*` … `*/`	none

Table 7.1: Token classes recognised by the lexer

Keywords are recognised after an identifier is scanned: the lexeme is upper-cased and compared with the keyword list (`check_keyword`). Three inputs raise a lexical error: an unknown character such as `@`, a lone `!`, and an unterminated string.

7.2 Syntax Analysis

The parser (`sql_parser.c`) is a hand-written recursive-descent (LL(1)) parser: one function per non-terminal, deciding each step from the current token only. The grammar in EBNF is:

```
query         → select_clause from_clause [where_clause]
                [order_clause] [limit_clause] [";"] EOF
select_clause → "SELECT" ( "*" | IDENT { "," IDENT } )
from_clause   → "FROM" IDENT
where_clause  → "WHERE" or_expr
or_expr       → and_expr { "OR" and_expr }
and_expr      → primary { "AND" primary }
primary       → "(" or_expr ")" | IDENT comp_op literal
comp_op       → "=" | "!=" | "<>" | ">" | ">=" | "<" | "<="
literal       → NUMBER | STRING
order_clause  → "ORDER" "BY" IDENT [ "ASC" | "DESC" ]
limit_clause  → "LIMIT" NUMBER
```

Precedence comes from the layering: `primary` binds tightest, then `AND`, then `OR`. The `{ … }` repetitions are written as `while` loops that fold operands to the left, so `a AND b AND c` becomes `(a AND b) AND c`. With the `--tree` flag the parser prints the concrete parse tree:

```
<Query>
 ├── <SelectClause>
 │    ├── Terminal: 'SELECT'
 │    └── <ProjectionList>
 │         ├── <Identifier: "name">
 │         ├── Terminal: ','
 │         └── <Identifier: "age">
 ├── <FromClause>
 │    ├── Terminal: 'FROM'
 │    └── <CollectionIdentifier: "students">
 ├── <WhereClause>
 │    ├── Terminal: 'WHERE'
 │    └── <ConditionExpression>
 │         └── <ComparisonExpr>
 │              ├── <FieldIdentifier: "age">
 │              ├── <ComparisonOp: ">=">
 │              └── <IntegerLiteral: 20>
 ├── <OrderByClause>
 │    ├── Terminal: 'ORDER'
 │    ├── Terminal: 'BY'
 │    ├── <FieldIdentifier: "age">
 │    └── Terminal: 'DESC'
 ├── <LimitClause>
 │    ├── Terminal: 'LIMIT'
 │    └── <IntegerLiteral: 3>
 └── Terminal: ';'
```

On a syntax error the parser records the first error only and reports the expected construct, for example:

```
Syntax error at [Line 4, Col 1]: Expected collection/table name identifier,
but found 'WHERE' (KEYWORD(WHERE))
```

7.3 Abstract Syntax Tree

The AST (`sql_ast.h`) drops punctuation and keeps meaning. The root `ASTQuery` holds the collection name, the projection list (or a wildcard flag), an optional `WHERE` tree, and the `ORDER BY` and `LIMIT` settings. The `WHERE` tree is built from `ASTFilter` nodes of two kinds:

- **Binary node:** a field, a comparison operator and a typed literal (`INT`, `FLOAT` or `STRING`).
- **Logical node:** `AND` or `OR` with left and right children.

Filter nodes are allocated on the heap and freed recursively by `ast_free` after compilation. The `--ast` output for the running example:

```
SelectQueryNode
  [Projections]: name, age
  [From Collection]: students
  [Where Filter]:
    BinaryCondition: [age >= 20]
  [Order By]: age (DESCENDING)
  [Limit]: 3
```

7.4 Symbol Table and Semantic Analysis

The schema catalog (`schema_table.c`) is the symbol table. Each entry is a field symbol scoped to its collection, with a data type and storage width. It holds 2 collections and 10 symbols.

Collection	Field	Type	Width
students	name, city, status	STRING	64 B
students	age	INT	4 B
students	gpa	FLOAT	8 B
employees	name, department, role	STRING	64 B
employees	experience	INT	4 B
employees	salary	FLOAT	8 B

Table 7.2: Symbol table (schema catalog)

Lookups are case-insensitive. The semantic analyser walks the AST and applies four checks in order, stopping at the first failure:

1. The collection in `FROM` exists.
2. Every column in `SELECT` exists in that collection.
3. Every field in `WHERE` exists, and its type matches the literal: numeric fields take number literals, string fields take string literals.

4. The `ORDER BY` field exists.

For example, `WHERE age = 'twenty'` fails with `Semantic Type Error: Cannot compare numeric field 'age' with string literal 'twenty'`.

7.5 Intermediate Code Generation

The IR generator (`tac.c`) produces three-address code, then derives quadruples and triples from it. It works in two steps:

1. **Condition code.** A post-order walk of the `WHERE` tree gives each comparison a new temporary, then combines child temporaries at each `AND` or `OR` node.
2. **Operator chain.** Relational operators are chained in a fixed order, `SCAN` → `FILTER` → `SORT` → `LIMIT` → `PROJECT`, each consuming the previous temporary. Absent clauses are skipped.

For test case 2 (`SELECT name, department, salary FROM employees WHERE salary >= 70000 AND department = 'Engineering' ORDER BY salary DESC`) the output is:

```
(1)  t1 = SCAN(employees)
(2)  t2 = salary >= 70000
(3)  t3 = department == 'Engineering'
(4)  t4 = t2 AND t3
(5)  t5 = FILTER(t1, t4)
(6)  t6 = SORT(t5, salary (DESC))
(7)  t7 = PROJECT(t6, [name,department,salary])
```

Quadruples store the result temporary explicitly. Triples drop it, and later instructions refer to an earlier result by its index, such as `(4)`.

		Quadruple			Triple	
#	**Op**	**Arg1**	**Arg2**	**Result**	**Arg1**	**Arg2**
1	SCAN	employees	–	t1	employees	–
2	>=	salary	70000	t2	salary	70000
3	==	department	’Engineering’	t3	department	’Engineering’
4	AND	t2	t3	t4	(2)	(3)
5	FILTER	t1	t4	t5	(1)	(4)
6	SORT	t5	salary (DESC)	t6	(5)	salary (DESC)
7	PROJECT	t6	name,department,salary	t7	(6)	name,department,salary

Table 7.3: Quadruples and triples for test case 2

7.6 Code Optimization

The optimizer (`optimizer.c`) runs after IR generation and reports which optimizations apply to the query. Two are built into the shape of the IR and of the generated pipeline:

- **Predicate pushdown.** `FILTER` always comes straight after `SCAN`, so `$match` is always the first pipeline stage. Documents are discarded before the $O(N \log N)$ sort instead of after it.
- **Projection pruning.** A `$project` stage is emitted only when named columns are requested, and it is placed last, as MongoDB's optimization guide recommends [7]. For `SELECT *` the stage is omitted altogether because it would keep every field.

Literal types (`INT`, `FLOAT`, `STRING`) are resolved and checked at compile time, so no type conversion is needed at run time. The grammar has no arithmetic expressions yet, so constant folding currently has nothing to fold.

```
1. [Predicate Pushdown]       : ENABLED & APPLIED (Filter executed before sort/projection)
2. [Projection Column Pruning]: APPLIED (2 attributes selected)
3. [Compile-Time Folding]     : APPLIED (Static type and literal resolution)
4. [Dead Stage Elimination]   : 0 redundant stages detected
```

7.7 Target Code Generation

The code generator (`mongo_codegen.c`) walks the AST and emits a `db.<collection>.aggregate([...])` call. Stages appear in the order `$match`, `$sort`, `$limit`, `$project`, and only for clauses present in the query.

Filters are emitted recursively. A comparison becomes `{ field: value }` for equality or `{ field: { $op: value } }` otherwise. A logical node becomes `{ $and: [left, right] }` or `{ $or: [left, right] }`. Because the AST already encodes precedence, nested conditions come out correctly:

```
SELECT name FROM students
WHERE (age > 19 AND gpa >= 3.5) OR status = 'Graduated'
ORDER BY gpa DESC;
```

```
db.students.aggregate([
  { "$match": { "$or": [
      { "$and": [ { "age": { "$gt": 19 } }, { "gpa": { "$gte": 3.50 } } ] },
      { "status": "Graduated" }
  ] } },
  { "$sort": { "gpa": -1 } },
  { "$project": { "_id": 0, "name": 1 } }
]);
```

7.8 Interpretation and Execution

The execution engine (`engine.c`) is a tree-walking interpreter over the AST. It loads the collection's documents into memory, evaluates the `WHERE` tree recursively for each document, sorts the matches with `qsort` and a field comparator, applies the limit and prints the projected fields as JSON. The query above returns Diana Prince (GPA 3.95), Alice Walker (3.85) and Evan Wright (Graduated, 3.10), which is the correct answer for the sample data in Appendix A.3.

7.9 Error Detection and Recovery

Errors are caught in the earliest phase that can see them. The compiler uses a stop-at-first-error strategy: it reports the fault with its position and halts with a phase-specific exit code.

Phase	Error	Example input	Message	Exit
Lexical	Unknown character	`WHERE age @ 20`	`Lexical error at [Line 1, Col 37]: Unrecognized or malformed token '@'`	1
Lexical	Unterminated string	`WHERE city = 'Seattle;`	`Lexical error at [Line 1, Col 40]: … malformed token 'Seattle;'`	1
Syntax	Missing table name	`FROM WHERE age >= 20`	`Expected collection/table name identifier, but found 'WHERE'`	1
Syntax	Unbalanced parenthesis	`WHERE (age > 19;`	`Expected closing parenthesis ')', but found ';'`	1
Semantic	Unknown collection	`FROM teachers`	`Collection 'teachers' does not exist in schema catalog`	2
Semantic	Unknown column	`SELECT name, nonexistent_column`	`Column 'nonexistent_column' in SELECT clause is not defined`	2
Semantic	Type mismatch	`WHERE city = 25`	`Cannot compare string field 'city' with numeric literal`	2

Table 7.4: Error detection by phase

Chapter 8: Implementation

SQL2Mongo is 2,370 lines of C99 in ten modules, using only the C standard library. It builds with GCC under `-Wall -Wextra` with no warnings.

8.1 Tools Used

Item	Used
Language	C99
Compiler	GCC 6.3.0 (MinGW) on Windows 11
Libraries	C standard library only (`stdio`, `stdlib`, `string`, `ctype`, `stdbool`)
Editor	Visual Studio Code
Version control	Git
Test automation	PowerShell script `test_all.ps1`

Table 8.1: Implementation tools

8.2 Modules

Module	Lines	Responsibility	Key functions
sql_lexer	372	Scanner, token types, token table	lexer_next_token, check_keyword, lexer_print_all
sql_parser	395	Recursive-descent parser, parse-tree printer	parser_parse, parse_or_expr, parse_and_expr, parse_primary_filter
sql_ast	223	AST node types, constructors, printer, cleanup	ast_create_binary_filter, ast_create_logical_filter, ast_free
schema_table	236	Symbol table and semantic checks	schema_init, schema_get_field_type, schema_validate_query
tac	257	Three-address code, quadruples, triples	tac_generate, quads_print, triples_print
optimizer	59	Optimization pass and report	optimizer_run, optimizer_print_report
query_plan	129	Relational query plan with per-operator cost estimates (not yet exposed by a command-line flag)	plan_generate, plan_print
mongo_codegen	137	MQL aggregation-pipeline generator	mongo_codegen_pipeline, emit_filter_json
engine	320	In-memory document store and query interpreter	engine_execute_query, eval_filter, doc_sort_compare
main	242	Command-line driver that runs the phases in order	main, read_entire_file
Total	**2,370**		

Table 8.2: Source modules (lines counted over the `.c` and `.h` files)

8.3 Key Algorithms

- **Scanning:** a character-class switch with one-character lookahead and maximal munch; keywords found by upper-casing the identifier and comparing it with the keyword list. Linear in input length.
- **Parsing:** LL(1) recursive descent with one token of lookahead; left-associative operator chains built by loops. Linear in the number of tokens.
- **Semantic analysis:** recursive AST walk with linear symbol lookup in the catalog (10 symbols).

- **IR generation:** post-order traversal of the filter tree; a counter supplies fresh temporaries `t1, t2, ….` Triples are derived by replacing each temporary with the index of the quadruple that produced it.
- **Code generation:** recursive emission of nested JSON for `$and` and `$or`.
- **Execution:** per-document recursive evaluation of the filter tree, `qsort` for `ORDER BY`, then limit and projection.

8.4 Project Structure

```
compiler-project/
├── build.bat            one-step GCC build
├── test_all.ps1         automated test suite
├── data/                sample collections (students.json, employees.json)
├── examples/            test queries (4 valid, 2 error cases)
├── report/              this report (LaTeX source and PDF)
└── src/                 ten modules listed in Section 8.2
```

8.5 User Instructions

Build with `build.bat`, or directly:

```
gcc -std=c99 -Wall -Wextra src/*.c -o sql2mongo.exe
```

Run on a query file or an inline query. With no flag, all phases run (`--all`).

```
sql2mongo.exe examples/query1_basic.sql --all
sql2mongo.exe "SELECT name, age FROM students WHERE age >= 20 ORDER BY age DESC
    LIMIT 2;" --mongo
```

Flag	Shows
`--tokens`	Token table (lexical analysis)
`--tree`	Concrete parse tree
`--ast`	Abstract syntax tree
`--schema`	Symbol table and semantic check result
`--tac`, `--quads`, `--triples`	Three-address code, quadruples, triples
`--opt`	Optimization report
`--mongo`	Generated MongoDB pipeline
`--run`	Query results from the execution engine
`--all`	Every phase in order (default)

Table 8.3: Command-line flags

Run the automated test suite with:

```
powershell -ExecutionPolicy Bypass -File .\test_all.ps1
```

Chapter 9: Testing and Results

Of 19 test queries, 17 behave exactly as expected. The other 2 run correctly in the engine but produce MQL that a real MongoDB server would treat differently (Section 9.4). All tests were run on 23 October 2026 with a fresh build of the current source.

9.1 Test Cases

Tests cover valid, boundary, lexical-error, syntax-error and semantic-error inputs. Cases 1–4, 11 and 15 form the automated suite in `test_all.ps1`; all six return their expected exit codes.

Table 9.1: Test cases and results

#	Category	Input (query or clause)	Expected	Actual	Status
1	Valid	`SELECT name, age FROM students WHERE age >= 20 ORDER BY age DESC LIMIT 3;`	Exit 0; 3 documents, oldest first	Diana (23), Alice (22), Bob (21)	Pass
2	Valid, `AND`	`SELECT name, department, salary FROM employees WHERE salary >= 70000 AND department = 'Engineering' ORDER BY salary DESC;`	Exit 0; `$and` filter	Sarah Connor (95000), John Doe (72000)	Pass
3	Valid, `*` and float	`SELECT * FROM students WHERE gpa >= 3.85;`	Exit 0; no `$project` stage	3 full documents; pipeline has `$match` only	Pass
4	Valid, `OR`	`SELECT name, city FROM students WHERE city = 'Seattle' OR city = 'Boston' ORDER BY name ASC;`	Exit 0; `$or` filter, sorted by name	Alice, Bob, Charlie	Pass
5	Complex	`WHERE (age > 19 AND gpa >= 3.5) OR status = 'Graduated' ORDER BY gpa DESC`	`$and` nested inside `$or`	Correct nesting; Diana, Alice, Evan	Pass
6	Boundary	`WHERE age > 100`	Exit 0; empty result	0 documents returned	Pass
7	Boundary	`select NAME, Age from STUDENTS where AGE < 21 limit 1` (mixed case, no `;`)	Exit 0; 1 document	Charlie Brown returned; MQL keeps the user's casing	Partial
8	Boundary	`LIMIT 0`	Rejected, or valid MQL	Accepted; emits `$limit: 0`	Partial

#	Category	Input (query or clause)	Expected	Actual	Status
9	Lexical error	`WHERE age @ 20`	Exit 1	Exit 1, unrecognised token `'@'` at col 37	Pass
10	Lexical error	`WHERE city = 'Seattle;`	Exit 1	Exit 1, unterminated string reported	Pass
11	Syntax error	`SELECT name, age FROM WHERE age >= 20;`	Exit 1	Exit 1, expected table name, at line 4	Pass
12	Syntax error	`ORDER age`	Exit 1	Exit 1, expected `BY` after `ORDER`	Pass
13	Syntax error	`LIMIT 2 extra;`	Exit 1	Exit 1, unexpected trailing tokens	Pass
14	Syntax error	`WHERE (age > 19;`	Exit 1	Exit 1, expected closing parenthesis	Pass
15	Semantic error	`SELECT name, nonexistent_column FROM students WHERE age = 'twenty';`	Exit 2	Exit 2, unknown column reported first	Pass
16	Semantic error	`WHERE age = 'twenty'`	Exit 2	Exit 2, numeric field compared with string	Pass
17	Semantic error	`WHERE city = 25`	Exit 2	Exit 2, string field compared with number	Pass
18	Semantic error	`FROM teachers`	Exit 2	Exit 2, collection does not exist	Pass
19	Semantic error	`FROM students ORDER BY salary`	Exit 2	Exit 2, `ORDER BY` field does not exist	Pass

9.2 Output Evidence

Token table for test case 1 (`--tokens`):

```
+----+----------------------+---------------------+------+------+
| #  | Token Type           | Lexeme              | Line | Col  |
+----+----------------------+---------------------+------+------+
| 1  | KEYWORD(SELECT)      | SELECT              | 2    | 1    |
| 2  | IDENTIFIER           | name                | 2    | 8    |
| 3  | COMMA(,)             | ,                   | 2    | 12   |
| 4  | IDENTIFIER           | age                 | 2    | 14   |
| 5  | KEYWORD(FROM)        | FROM                | 3    | 1    |
| 6  | IDENTIFIER           | students            | 3    | 6    |
| 7  | KEYWORD(WHERE)       | WHERE               | 4    | 1    |
| 8  | IDENTIFIER           | age                 | 4    | 7    |
| 9  | OPERATOR(>=)         | >=                  | 4    | 11   |
| 10 | NUMBER               | 20                  | 4    | 14   |
| 11 | KEYWORD(ORDER)       | ORDER               | 5    | 1    |
```

```
| 12 | KEYWORD(BY)           | BY                  | 5    | 7    |
| 13 | IDENTIFIER            | age                 | 5    | 10   |
| 14 | KEYWORD(DESC)         | DESC                | 5    | 14   |
| 15 | KEYWORD(LIMIT)        | LIMIT               | 6    | 1    |
| 16 | NUMBER                | 3                   | 6    | 7    |
| 17 | SEMICOLON(;)          | ;                   | 6    | 8    |
+----+-----------------------+---------------------+------+------+
Total Tokens Scanned: 17 | Lexical Errors: 0 | Status: PASS
```

Generated pipeline for test case 1 (`--mongo`):

```
db.students.aggregate([
  { "$match": { "age": { "$gte": 20 } } },
  { "$sort": { "age": -1 } },
  { "$limit": 3 },
  { "$project": { "_id": 0, "name": 1, "age": 1 } }
]);
```

Execution results for test case 1 (`--run`):

```
Database: local_db | Collection: students | Evaluated: 5 | Returned: 3
{ "name": "Diana Prince", "age": 23 }
{ "name": "Alice Walker", "age": 22 }
{ "name": "Bob Smith", "age": 21 }
```

9.3 Performance

A complete seven-phase run averaged 48.8 ms for test case 1 and 53.8 ms for test case 2, over 100 runs each. A run that only prints the help text averaged 30.7 ms, so process start-up dominates. The compiler phases and their console output account for about 18–23 ms.

Memory use is fixed and small because every structure has a static size:

Structure	**Size**
Token	152 B
AST root (`ASTQuery`)	2,200 B, plus 232 B per filter node
IR program (TAC, quadruples, triples)	14,352 B
One engine document	3,336 B
Engine buffers (two arrays of 32 documents)	about 213 KB

Table 9.2: Memory footprint of the main data structures

Fixed limits: 32 projected columns, 32 IR instructions, 128-character lexemes, 32 documents per collection.

9.4 Findings

- **Case 7:** keywords and schema lookups are case-insensitive, but the generator copies identifiers as typed. MongoDB is case-sensitive, so `db.STUDENTS` and `"AGE"` would match nothing on a real server. Identifiers should be normalised to their catalog spelling.
- **Case 8:** MongoDB's `$limit` stage takes a positive integer [8], so `LIMIT 0` should be rejected during semantic analysis.

Chapter 10: Conclusion

SQL2Mongo meets all eight objectives: it compiles a useful subset of SQL into valid MongoDB aggregation pipelines through seven hand-built phases, and it catches lexical, syntax and semantic errors before any code is generated.

The project started from a practical problem I ran into during my internship: knowing SQL well, but having to write MongoDB queries. SQL2Mongo removes the need to translate each query by hand. Its phase-by-phase output also shows how each SQL clause maps to its MongoDB form, so it doubles as a learning aid for anyone making the same switch.

The project shows that the classic compiler pipeline carries over directly to query translation. Lexing, recursive-descent parsing, symbol tables, three-address code and code generation each have a clear job, and the AST holds them together. The visible output of every phase made the compiler easy to debug and is the core of its demonstration.

Testing with 19 queries confirmed correct translation and execution for valid input, and precise diagnostics for invalid input. It also found two gaps in target-code fidelity, identifier casing and `LIMIT 0`, which are the first items in Chapter 11.

Chapter 11: Future Enhancements

1. **Target-code fidelity:** rewrite identifiers to their catalog spelling, and reject `LIMIT` values below 1.
2. **Data-driven schema:** load collections from `data/*.json` and infer the symbol table from them, instead of the built-in catalog and documents.
3. **Richer filters:** `NOT`, `IN` (`$in`), `LIKE` (`$regex`), `BETWEEN` (a `$gte`/`$lte` pair) and `OFFSET` (`$skip`).
4. **Aggregation and joins:** `GROUP BY` with `COUNT`, `SUM` and `AVG` (`$group`), and `JOIN` (`$lookup`).
5. **Real IR optimizations:** flatten chains such as `a AND b AND c` into one `$and` array, merge range conditions on the same field into one operator object, and drop redundant predicates.
6. **Error recovery:** panic-mode recovery in the parser, so one run can report several syntax errors instead of only the first.
7. **Query plan view:** expose the existing `query_plan` module through a `--plan` flag.
8. **Live validation:** run generated pipelines on a MongoDB server through the official C driver and compare the results with the built-in engine.

References

[1] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston: Pearson Addison-Wesley, 2006.

[2] E. F. Codd, “A relational model of data for large shared data banks,” *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[3] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie and T. G. Price, “Access path selection in a relational database management system,” in *Proc. ACM SIGMOD*, 1979, pp. 23–34.

[4] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Englewood Cliffs: Prentice Hall, 1988.

[5] MongoDB, Inc., “SQL to Aggregation Mapping Chart,” MongoDB Database Manual. https://www.mongodb.com/docs/manual/reference/sql-aggregation-comparison/

[6] MongoDB, Inc., “Aggregation Pipeline,” MongoDB Database Manual. https://www.mongodb.com/docs/manual/core/aggregation-pipeline/

[7] MongoDB, Inc., “Aggregation Pipeline Optimization,” MongoDB Database Manual. https://www.mongodb.com/docs/manual/core/aggregation-pipeline-optimization/

[8] MongoDB, Inc., “$limit (aggregation stage),” MongoDB Database Manual. https://www.mongodb.com/docs/manual/reference/operator/aggregation/limit/

[9] MongoDB, Inc., “mongotranslate SQL Translation Tool,” MongoDB BI Connector documentation. https://www.mongodb.com/docs/bi-connector/current/reference/mongotranslate/

[10] V. Russell, “sql-to-mongo-db-query-converter,” GitHub repository. https://github.com/vincentrussell/sql-to-mongo-db-query-converter

[11] *Compiler Design Laboratory Project Instruction Manual*, BCSE307P, Fall Semester 2026–27.

Web sources accessed 23 October 2026.

Appendix A: Code Excerpts and Sample Data

A.1 Operator Precedence in the Parser

From `sql_parser.c`. `OR` calls `AND`, which calls `primary`, so `AND` binds tighter. Each loop folds operands to the left.

```
static ASTFilter* parse_and_expr(Parser* p) {
    ASTFilter* left = parse_primary_filter(p);
    if (!left || p->has_error) return left;

    while (match(p, TOK_AND)) {
        ASTFilter* right = parse_primary_filter(p);
        if (!right || p->has_error) return left;
        left = ast_create_logical_filter(LOGIC_AND, left, right);
    }
    return left;
}

static ASTFilter* parse_or_expr(Parser* p) {
    ASTFilter* left = parse_and_expr(p);
    if (!left || p->has_error) return left;

    while (match(p, TOK_OR)) {
        ASTFilter* right = parse_and_expr(p);
        if (!right || p->has_error) return left;
        left = ast_create_logical_filter(LOGIC_OR, left, right);
    }
    return left;
}
```

A.2 Post-order Three-Address Code for Filters

From `tac.c`. Both children are translated first; their temporaries become the operands of the parent instruction.

```
} else if (f->type == FILTER_LOGICAL) {
    char left_t[32];
    char right_t[32];
    generate_filter_tac(p, f->left, left_t, sizeof(left_t));
    generate_filter_tac(p, f->right, right_t, sizeof(right_t));

    snprintf(out_res, sz, "t%d", ++p->temp_count);

    const char* log_op = (f->logical_op == LOGIC_AND) ? "AND" : "OR";
    char tac_line[128];
```

```
    snprintf(tac_line, sizeof(tac_line), "%s = %s %s %s",
             out_res, left_t, log_op, right_t);

    add_tac_quad(p, log_op, left_t, right_t, out_res, tac_line);
}
```

A.3 Sample Data

All test results in Chapter 9 can be checked against these two collections.

name	age	gpa	city	status
Alice Walker	22	3.85	Seattle	Active
Bob Smith	21	3.40	Boston	Active
Charlie Brown	19	3.90	Seattle	Probation
Diana Prince	23	3.95	New York	Active
Evan Wright	20	3.10	Chicago	Graduated

Table A.1: The `students` collection

name	department	salary	experience	role
Sarah Connor	Engineering	95000	6	Lead Dev
John Doe	Engineering	72000	3	Software Engineer
Jane Smith	Marketing	65000	4	Marketing Lead
Michael Scott	Management	85000	10	Regional Manager
Dwight Schrute	Sales	58000	5	Assistant Regional Manager

Table A.2: The `employees` collection